

Chapter 6

Methods: A Deeper Look

Java How to Program, 11/e

Questions? E-mail paul.deitel@deitel.com

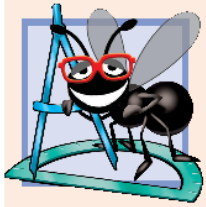
6.3 static Methods, static Fields and Class Math

- ▶ Sometimes a method performs a task that does not depend on an object.
 - Applies to the class in which it's declared as a whole
 - Known as a `static` method or a `class method`
- ▶ It's common for classes to contain convenient `static` methods to perform common tasks.
- ▶ To declare a method as `static`, place the keyword `static` before the return type in the method's declaration.
- ▶ Calling a `static` method
 - *`ClassName.methodName(arguments)`*

6.3 static Methods, static Fields and Class Math (Cont.)

Math Class Methods

- ▶ Class Math provides a collection of static methods that enable you to perform common mathematical calculations.
- ▶ Method arguments may be constants, variables or expressions.



Software Engineering Observation 6.4

Class `Math` is part of the `java.lang` package, which is implicitly imported by the compiler, so it's not necessary to import class `Math` to use its methods.

Method	Description	Example
<code>abs(x)</code>	absolute value of x	<code>abs(23.7)</code> is 23.7 <code>abs(0.0)</code> is 0.0 <code>abs(-23.7)</code> is 23.7
<code>ceil(x)</code>	rounds x to the smallest integer not less than x	<code>ceil(9.2)</code> is 10.0 <code>ceil(-9.8)</code> is -9.0
<code>cos(x)</code>	trigonometric cosine of x (x in radians)	<code>cos(0.0)</code> is 1.0
<code>exp(x)</code>	exponential method e^x	<code>exp(1.0)</code> is 2.71828 <code>exp(2.0)</code> is 7.38906
<code>floor(x)</code>	rounds x to the largest integer not greater than x	<code>floor(9.2)</code> is 9.0 <code>floor(-9.8)</code> is -10.0

Fig. 6.2 | Math class methods. (Part 1 of 2.)

Method	Description	Example
<code>log(x)</code>	natural logarithm of x (base e)	<code>log(Math.E)</code> is 1.0 <code>log(Math.E * Math.E)</code> is 2.0
<code>max(x, y)</code>	larger value of x and y	<code>max(2.3, 12.7)</code> is 12.7 <code>max(-2.3, -12.7)</code> is -2.3
<code>min(x, y)</code>	smaller value of x and y	<code>min(2.3, 12.7)</code> is 2.3 <code>min(-2.3, -12.7)</code> is -12.7
<code>pow(x, y)</code>	x raised to the power y (i.e., x^y)	<code>pow(2.0, 7.0)</code> is 128.0 <code>pow(9.0, 0.5)</code> is 3.0
<code>sin(x)</code>	trigonometric sine of x (x in radians)	<code>sin(0.0)</code> is 0.0
<code>sqrt(x)</code>	square root of x	<code>sqrt(900.0)</code> is 30.0
<code>tan(x)</code>	trigonometric tangent of x (x in radians)	<code>tan(0.0)</code> is 0.0

Fig. 6.2 | Math class methods. (Part 2 of 2.)

6.3 static Methods, static Fields and Class Math (Cont.)

- ▶ Recall that each object of a class maintains its *own* copy of every instance variable of the class.
- ▶ There are variables for which each object of a class does *not* need its own separate copy (as you'll see momentarily).
- ▶ Such variables are declared static and are also known as **class variables**.
- ▶ When objects of a class containing static variables are created, all the objects of that class share one copy of those variables.
- ▶ Together a class's static variables and instance variables are known as its **fields**.
- ▶ You'll learn more about static fields in Section 8.11.

6.3 static Methods, static Fields and Class Math (Cont.)

Math Class static Constants PI and E

- ▶ Math fields for commonly used mathematical constants
 - `Math.PI` (3.141592653589793)
 - `Math.E` (2.718281828459045)
- ▶ Declared in class Math with the modifiers `public`, `final` and `static`
 - `public` allows you to use these fields in your own classes.
 - A field declared with keyword `final` is *constant*—its value cannot change after the field is initialized.

6.3 static Methods, static Fields and Class Math (Cont.)

Why is method `main` declared `static`?

- ▶ The JVM attempts to invoke the `main` method of the class you specify—at this point no objects of the class have been created.
- ▶ Declaring `main` as `static` allows the JVM to invoke `main` without creating an instance of the class.

6.4 Declaring Methods with Multiple Parameters

- ▶ Multiple parameters are specified as a comma-separated list.
- ▶ There must be one argument in the method call for each parameter (sometimes called a **formal parameter**) in the method declaration.
- ▶ Each argument must be consistent with the type of the corresponding parameter.

```
1 // Fig. 6.3: MaximumFinder.java
2 // Programmer-declared method maximum with three double parameters.
3 import java.util.Scanner;
4
5 public class MaximumFinder {
6     public static void main(String[] args) {
7         // create Scanner for input from command window
8         Scanner input = new Scanner(System.in);
9
10        // prompt for and input three floating-point values
11        System.out.print(
12            "Enter three floating-point values separated by spaces: ");
13        double number1 = input.nextDouble(); // read first double
14        double number2 = input.nextDouble(); // read second double
15        double number3 = input.nextDouble(); // read third double
16
17        // determine the maximum value
18        double result = maximum(number1, number2, number3);
19
20        // display maximum value
21        System.out.println("Maximum is: " + result);
22    }
```

Fig. 6.3 | Programmer-declared method `maximum` with three `double` parameters. (Part 1 of 3.)

```
23
24 // returns the maximum of its three double parameters
25 public static double maximum(double x, double y, double z) {
26     double maximumValue = x; // assume x is the largest to start
27
28     // determine whether y is greater than maximumValue
29     if (y > maximumValue) {
30         maximumValue = y;
31     }
32
33     // determine whether z is greater than maximumValue
34     if (z > maximumValue) {
35         maximumValue = z;
36     }
37
38     return maximumValue;
39 }
40 }
```

Fig. 6.3 | Programmer-declared method `maximum` with three `double` parameters. (Part 2 of 3.)

```
Enter three floating-point values separated by spaces: 9.35 2.74 5.1  
Maximum is: 9.35
```

```
Enter three floating-point values separated by spaces: 5.8 12.45 8.32  
Maximum is: 12.45
```

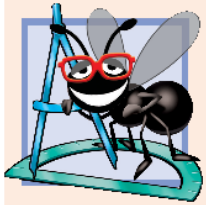
```
Enter three floating-point values separated by spaces: 6.46 4.12 10.54  
Maximum is: 10.54
```

Fig. 6.3 | Programmer-declared method `maximum` with three `double` parameters. (Part 3 of 3.)



Software Engineering Observation 6.5

Methods can return at most one value, but the returned value could be a reference to an object that contains many values in its instance variables.



Software Engineering Observation 6.6

Variables should be declared as fields only if they're required for use in more than one method of the class or if the program should save their values between calls to the class's methods.



Common Programming Error 6.1

Declaring method parameters of the same type as `float x, y` instead of `float x, float y` is a syntax error—a type is required for each parameter in the parameter list.

6.4 Declaring Methods with Multiple Parameters (Cont.)

Implementing method `maximum` by reusing method `Math.max`

- ▶ Two calls to `Math.max`, as follows:
 - `return Math.max(x, Math.max(y, z));`
- ▶ The first specifies arguments `x` and `Math.max(y, z)`.
- ▶ Before any method can be called, its arguments must be evaluated to determine their values.
- ▶ If an argument is a method call, the method call must be performed to determine its return value.
- ▶ The result of the first call is passed as the second argument to the other call, which returns the larger of its two arguments.

6.4 Declaring Methods with Multiple Parameters (Cont.)

Assembling Strings with String Concatenation

- ▶ **String concatenation**
 - Assemble `String` objects into larger strings with operators `+` or `+=`.
- ▶ When both operands of operator `+` are `Strings`, operator `+` creates a new `String` object
 - characters of the right operand are placed at the end of those in the left operand
- ▶ *Every primitive value and object in Java can be represented as a `String`.*
- ▶ When one of the `+` operator's operands is a `String`, the other is converted to a `String`, then the two are concatenated.
- ▶ If a boolean is concatenated with a `String`, the boolean is converted to the `String` `"true"` or `"false"`.
- ▶ All objects have a `toString` method that returns a `String` representation of the object.



Common Programming Error 6.2

It's a syntax error to break a `String` literal across lines. If necessary, you can split a `String` into several smaller `Strings` and use concatenation to form the desired `String`.



Common Programming Error 6.3

Confusing the `+` operator used for string concatenation with the `+` operator used for addition can lead to strange results. Java evaluates the operands of an operator from left to right. For example, if integer variable `y` has the value 5, the expression `"y + 2 = " + y + 2` results in the string `"y + 2 = 52"`, not `"y + 2 = 7"`, because first the value of `y` (5) is concatenated to the string `"y + 2 = "`, then the value 2 is concatenated to the new larger string `"y + 2 = 5"`. The expression `"y + 2 = " + (y + 2)` produces the desired result `"y + 2 = 7"`.

6.5 Notes on Declaring and Using Methods

- ▶ Three ways to call a method:
 - Using a method name by itself to call another method of the same class
 - Using a variable that contains a reference to an object, followed by a dot (.) and the method name to call a method of the referenced object
 - Using the class name and a dot (.) to call a `static` method of a class

6.5 Notes on Declaring and Using Methods (Cont.)

- ▶ Three ways to return control to the statement that calls a method:
 - When the program flow reaches the method-ending right brace
 - When the following statement executes
`return;`
 - When the method returns a result with a statement like
`return expression;`



Common Programming Error 6.4

Declaring a method outside the body of a class declaration or inside the body of another method is a syntax error.



Common Programming Error 6.5

Declaring a local variable in a method with the same name as one of the method's parameters is a compilation error.



Common Programming Error 6.6

Forgetting to return a value from a method that should return a value is a compilation error. If a return type other than `void` is specified, the method must contain a `return` statement that returns a value consistent with the method's return type. Returning a value from a method whose return type has been declared `void` is a compilation error.

6.5 Notes on Declaring and Using Methods (Cont.)

- ▶ Non-`static` methods are typically called `instance methods`.
- ▶ A `static` method can call other `static` methods of the same class directly and can manipulate `static` variables in the same class directly.
 - To access the class's instance variables and instance methods, a `static` method must use a reference to an object of the class.

6.6 Method-Call Stack and Activation Records

- ▶ **Stack** data structure
 - Analogous to a pile of dishes
 - A dish is placed on the pile at the top (referred to as **pushing** the dish onto the stack).
 - A dish is removed from the pile from the top (referred to as **popping** the dish off the stack).
- ▶ **Last-in, first-out (LIFO)** data structures
 - The *last* item pushed onto the stack is the *first* item popped from the stack.

6.6 Method-Call Stack and Activation Records (cont.)

Method-Call Stack

- ▶ One of the most important mechanisms for computer science students to understand is the **method-call stack** (or **program-execution stack**).
 - supports the method call/return mechanism.
 - Also supports the creation, maintenance and destruction of each called method's local variables.

6.6 Method-Call Stack and Activation Records (cont.)

Stack Frames

- ▶ Each method eventually must return control to the function that called it.
- ▶ Each time a method calls another method, an entry is pushed onto the method call stack.
 - This entry, called a **stack frame** or an **activation record**, contains the *return address* that the called method needs in order to return to the calling method.

6.6 Method-Call Stack and Activation Records (cont.)

Local Variables and Stack Frames

- ▶ When a method call returns, the stack frame for the method call is *popped*, and control transfers to the return address in the popped stack frame.

Stack Overflow

- ▶ The amount of memory in a computer is finite, so only a certain amount of memory can be used to store activation records on the method call stack.
- ▶ If more method calls occur than can have their activation records stored on the method call stack, an a fatal error known as **stack overflow** occurs.

6.7 Argument Promotion and Casting

- ▶ **Argument promotion**
 - Converting an argument's value, if possible, to the type that the method expects to receive in its corresponding parameter.
- ▶ Conversions may lead to compilation errors if Java's **promotion rules** are not satisfied.
- ▶ **Promotion rules**
 - specify which conversions are allowed.
 - apply to expressions containing values of two or more primitive types and to primitive-type values passed as arguments to methods.
- ▶ Each value is promoted to the “highest” type in the expression.
- ▶ Figure 6.4 lists the primitive types and the types to which each can be promoted.

Type	Valid promotions
double	None
float	double
long	float or double
int	long, float or double
char	int, long, float or double
short	int, long, float or double (but not char)
byte	short, int, long, float or double (but not char)
boolean	None (boolean values are not considered to be numbers in Java)

Fig. 6.4 | Promotions allowed for primitive types.

6.7 Argument Promotion and Casting (Cont.)

- ▶ Converting values to types lower in the table of Fig. 6.4 will result in different values if the lower type cannot represent the value of the higher type
- ▶ In cases where information may be lost due to conversion, the Java compiler requires you to use a cast operator to explicitly force the conversion to occur—otherwise a compilation error occurs.



Common Programming Error 6.7

Casting a primitive-type value to another primitive type may change the value if the new type is not a valid promotion. For example, casting a floating-point value to an integer value may introduce truncation errors (loss of the fractional part) into the result.

6.11 Scope of Declarations (Cont.)

- ▶ Basic scope rules:
 - The scope of a parameter declaration is the body of the method in which the declaration appears.
 - The scope of a local-variable declaration is from the point at which the declaration appears to the end of that block.
 - The scope of a local-variable declaration that appears in the initialization section of a for statement's header is the body of the for statement and the other expressions in the header.
 - A method or field's scope is the entire body of the class.
- ▶ *Any* block may contain variable declarations.
- ▶ If a local variable or parameter in a method has the same name as a field of the class, the field is *hidden* until the block terminates execution—this is called **shadowing**.

```
1  // Fig. 6.9: Scope.java
2  // Scope class demonstrates field and local-variable scopes.
3
4  public class Scope {
5      // field that is accessible to all methods of this class
6      private static int x = 1;
7
8      // method main creates and initializes local variable x
9      // and calls methods useLocalVariable and useField
10     public static void main(String[] args) {
11         int x = 5; // method's local variable x shadows field x
12
13         System.out.printf("local x in main is %d\n", x);
14
15         useLocalVariable(); // useLocalVariable has local x
16         useField(); // useField uses class Scope's field x
17         useLocalVariable(); // useLocalVariable reinitializes local x
18         useField(); // class Scope's field x retains its value
19
20         System.out.printf("%nlocal x in main is %d\n", x);
21     }
```

Fig. 6.9 | Scope class demonstrates field and local-variable scopes. (Part 1 of 3.)

```
22
23 // create and initialize local variable x during each call
24 public static void useLocalVariable() {
25     int x = 25; // initialized each time useLocalVariable is called
26
27     System.out.printf(
28         "%nlocal x on entering method useLocalVariable is %d%n", x);
29     ++x; // modifies this method's local variable x
30     System.out.printf(
31         "local x before exiting method useLocalVariable is %d%n", x);
32 }
33
34 // modify class Scope's field x during each call
35 public static void useField() {
36     System.out.printf(
37         "%nfield x on entering method useField is %d%n", x);
38     x *= 10; // modifies class Scope's field x
39     System.out.printf(
40         "field x before exiting method useField is %d%n", x);
41 }
42 }
```

Fig. 6.9 | Scope class demonstrates field and local-variable scopes. (Part 2 of 3.)

local x in main is 5

local x on entering method useLocalVariable is 25

local x before exiting method useLocalVariable is 26

field x on entering method useField is 1

field x before exiting method useField is 10

local x on entering method useLocalVariable is 25

local x before exiting method useLocalVariable is 26

field x on entering method useField is 10

field x before exiting method useField is 100

local x in main is 5

Fig. 6.9 | Scope class demonstrates field and local-variable scopes. (Part 3 of 3.)



Good Programming Practice 6.3

Declare variables as close to where they're first used as possible.

6.12 Method Overloading

- ▶ **Method overloading**
 - Methods of the same name declared in the same class
 - Must have different sets of parameters
- ▶ Compiler selects the appropriate method to call by examining the number, types and order of the arguments in the call.
- ▶ Used to create several methods with the same name that perform the same or similar tasks, but on different types or different numbers of arguments.
- ▶ Literal integer values are treated as type `int`, so the method call in line 9 invokes the version of `square` that specifies an `int` parameter.
- ▶ Literal floating-point values are treated as type `double`, so the method call in line 10 invokes the version of `square` that specifies a `double` parameter.

```
1  // Fig. 6.10: MethodOverload.java
2  // Overloaded method declarations.
3
4  public class MethodOverload {
5      // test overloaded square methods
6      public static void main(String[] args) {
7          System.out.printf("Square of integer 7 is %d\n", square(7));
8          System.out.printf("Square of double 7.5 is %f\n", square(7.5));
9      }
10
11     // square method with int argument
12     public static int square(int intValue) {
13         System.out.printf("\nCalled square with int argument: %d\n",
14             intValue);
15         return intValue * intValue;
16     }
17
18     // square method with double argument
19     public static double square(double doubleValue) {
20         System.out.printf("\nCalled square with double argument: %f\n",
21             doubleValue);
22         return doubleValue * doubleValue;
23     }
24 }
```

Fig. 6.10 | Overloaded method declarations. (Part 1 of 2.)

```
Called square with int argument: 7  
Square of integer 7 is 49
```

```
Called square with double argument: 7.500000  
Square of double 7.5 is 56.250000
```

Fig. 6.10 | Overloaded method declarations. (Part 2 of 2.)

6.12 Method Overloading (cont.)

Distinguishing Between Overloaded Methods

- ▶ The compiler distinguishes overloaded methods by their **signatures**—the methods' *name* and the *number, types* and *order* of its parameters.
- ▶ Return types of overloaded methods
 - *Method calls cannot be distinguished by return type.*
- ▶ Figure 6.10 illustrates the errors generated when two methods have the same signature and different return types.
- ▶ Overloaded methods can have different return types if the methods have different parameter lists.
- ▶ Overloaded methods need not have the same number of parameters.



Common Programming Error 6.8

Declaring overloaded methods with identical parameter lists is a compilation error regardless of whether the return types are different.